SÉANCE 2 SUR 3

PROGRAMMATION ORIENTÉE OBJET — JAVA

Héritage, interfaces, itérateurs, exceptions et patterns

Approfondir l’héritage, formaliser les contrats, parcourir des collections et gérer les erreurs avant le premier pattern.

À LA FIN DE CETTE SÉANCE

Objectifs pédagogiques

À la fin de cette séance, vous serez capable de…

1. Approfondir l'héritage, utiliser une classe abstraite et distinguer clairement classe abstraite et interface.
2. Rendre une classe triable avec Comparable ou selon plusieurs critères avec Comparator.
3. Parcourir une collection personnalisée avec Iterator et écrire votre propre classe Iterable.
4. Gérer les erreurs avec try / catch / finally et concevoir une exception personnalisée lorsque cela est pertinent.
5. Reconnaître lorsqu'un problème correspond à un design pattern connu, en commençant par Composite.

LIEN AVEC LA SÉANCE 1

Le socle construit en séance 1

- L’héritage (extends) construit une hiérarchie IS-A entre les classes et permet de redéfinir les comportements.
- Java n’autorise pas l’héritage multiple de classes : une classe ne peut étendre qu’une seule superclass.
- En revanche, une classe peut implémenter plusieurs interfaces : c’est notre point de départ.

extends, super et redéfinition

La séance 1 a introduit la relation IS-A et le polymorphisme. Ici, nous consolidons la construction d’une hiérarchie avant d’introduire les contrats abstraits.

Une subclass initialise d’abord la partie héritée avec super(...).

```
class Student extends Person {
    Student(String name) {
        super(name);
    }
}
```

Superclass construite en premier

Q2.1
Repérer l’appel de constructeur requis

Un type commun peut imposer un comportement

Une classe abstract représente un concept incomplet que l'on ne peut pas instancier directement.

Une méthode abstract déclare une obligation. Chaque subclass concrète fournit son implémentation.

```
abstract class Shape {
 abstract double area();
}
```

```
class Circle extends Shape {
double area() { return Math.PI * r * r; }
}
```

```
Shape s = new Circle(2);
s.area();
```

Une même opération, plusieurs implémentations

Le code client travaille avec le type commun. Chaque objet concret choisit sa propre implémentation à l'exécution.

@Override permet au compilateur de vérifier la signature de la méthode redéfinie.

```
List<Shape> shapes = List.of(
new Circle(2), new Rectangle(3, 4));
```

```
for (Shape s : shapes)
System.out.println(s.area());
```

Q2.2
Prédire la méthode exécutée

CONTRAT D’OBJECT

equals et hashCode définissent l’égalité logique

Toutes les classes héritent de java.lang.Object.

Si equals considère deux objets égaux, hashCode doit retourner la même valeur. Les collections fondées sur le hash dépendent de ce contrat.

```
a == b
même référence
```

```
a.equals(b)
même valeur logique
```

Q2.3 · Q2.4
Objets égaux → même hashCode

Interface ou classe abstraite

Une interface est avant tout un contrat : elle définit ce que les classes qui l'implémentent doivent fournir. Les interfaces Java modernes peuvent aussi contenir des méthodes default, static et private implémentées.

	Interface	Classe abstraite
Fields	constantes uniquement (public static final)	tous types de fields, y compris un état private
Corps des méthodes	aucun avant Java 8 ; default/static/private autorisés depuis	mélange de méthodes abstract et concrètes
Héritage multiple	une classe peut implémenter plusieurs interfaces	une classe ne peut étendre qu'une classe
Constructeurs	aucun	oui (appelés via super() depuis les subclasses)
À utiliser lorsque…	une capacité doit être partagée par des classes sans lien	un état ou une implémentation partielle doit être partagé

Déclarer et implémenter une interface

```
public interface Movable {
    void move(double dx, double dy);
    void reset();
}

public class Robot implements Movable {
    private double x, y;

    @Override
    public void move(double dx, double dy) { x += dx; y += dy; }

    @Override
    public void reset() { x = 0; y = 0; }
}

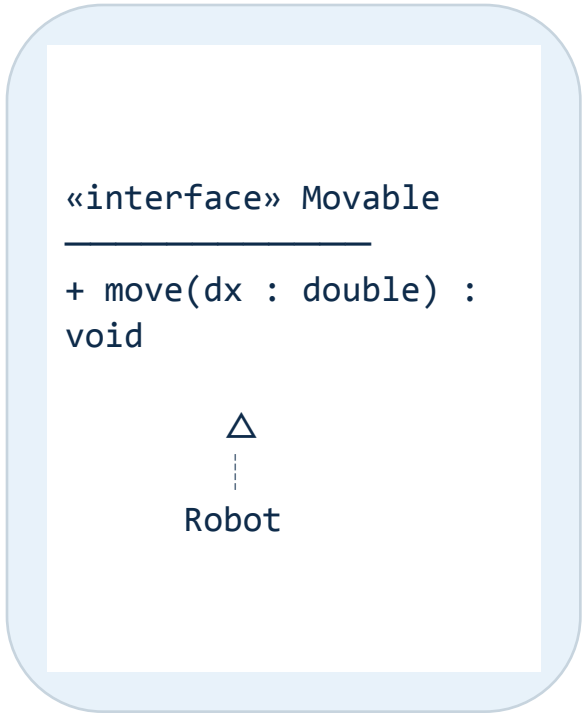
public interface Drawable {
    void draw();
}
// une classe peut implémenter plusieurs interfaces :
public class Robot implements Movable, Drawable { /* ... */ }
```

- **implements crée la relation CAN-DO, contrairement à la relation IS-A de extends.**
- Une interface peut étendre une autre interface et hériter de ses méthodes abstract.
- En UML : une ligne pointillée terminée par un triangle creux de la classe vers l'interface — réalisation.

La réalisation relie une classe à un contrat

En UML, une réalisation utilise une ligne discontinue terminée par un triangle creux orienté vers l'interface.

En Java, cette relation est exprimée avec implements.



⇄

```
interface Movable {
  void move(double dx);
}

class Robot
    implements Movable {
  public void move(double
dx) { }
}
```

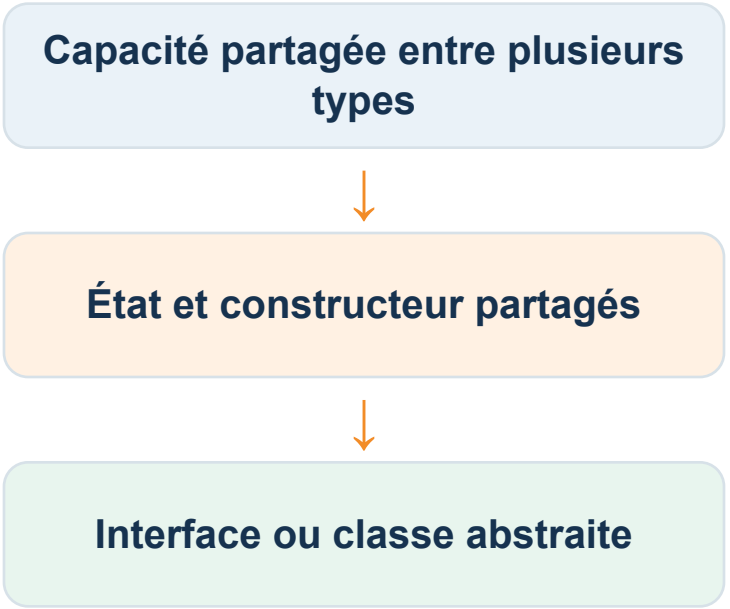
INTERFACE OU CLASSE ABSTRAITE

Les interfaces définissent des capacités ; les classes abstraites partagent un état

Une interface définit un contrat que des classes sans lien peuvent adopter.

Une classe abstraite peut partager des fields, des constructeurs et une implémentation partielle.

Une classe peut implémenter plusieurs interfaces mais n’étendre qu’une seule classe.



Les interfaces modernes peuvent fournir un comportement sans posséder d'état d'instance

Les fields d'une interface sont des constantes implicitement public, static et final.

Les méthodes abstract définissent le contrat. Les méthodes default et static peuvent fournir un comportement réutilisable ; les méthodes private facilitent la réutilisation interne.

La classe conserve la responsabilité de son état d'instance.

Méthode de la superclass prioritaire sur une méthode default

↓

Une seule méthode default est héritée normalement

↓

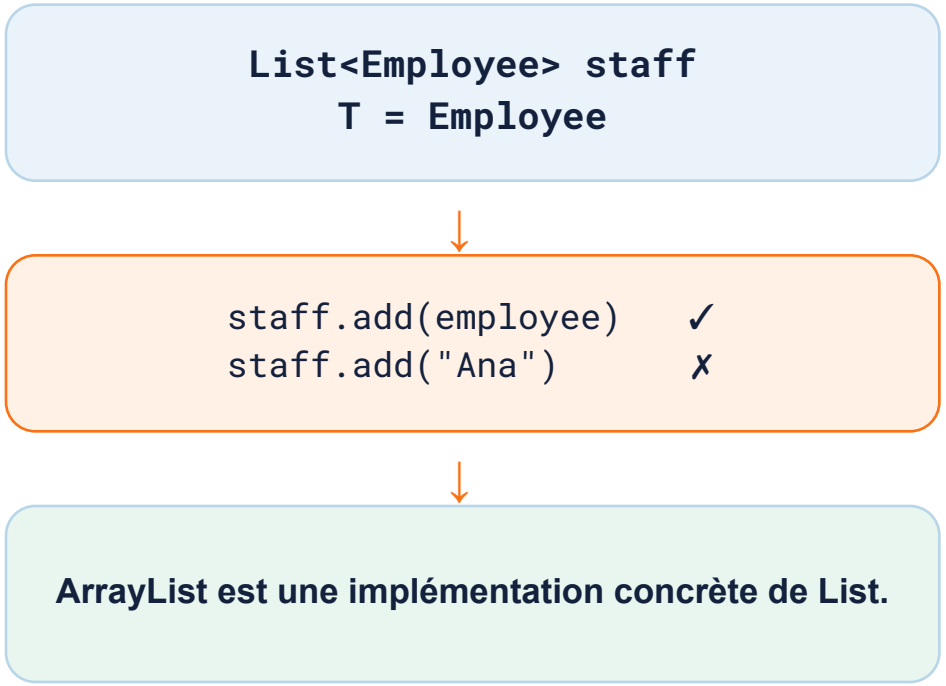
Méthodes default en conflit doivent être redéfinies explicitement

LIRE LE TYPE AVANT DE L'ÉTUDIER

List<Employee> signifie « une liste d'Employee »

Vous avez déjà utilisé des collections typées. La syntaxe complète des génériques sera expliquée en séance 4.

Pour l'instant, lisez T comme « le type des éléments » : le compilateur empêchera d'insérer un autre type.



Q3.1

Le tri commence par un contrat d’ordre

Arrays.sort fonctionne directement avec les types qui possèdent déjà un ordre naturel, comme String.

Votre propre classe doit implémenter Comparable ou être associée à un Comparator.



Q3.2

Comparable — Ordre naturel

```
public class Employee implements Comparable<Employee> {
    private String name;
    private double salary;
    // constructeur et getters omis

    @Override
    public int compareTo(Employee other) {
        return Double.compare(this.salary, other.salary);
    }
}

List<Employee> staff = new ArrayList<>(List.of(ana, leo,
mia));
Collections.sort(staff);          // utilise
automatiquement compareTo()
Employee lowest = Collections.min(staff);
```

- **compareTo retourne une valeur négative, nulle ou positive selon que l'objet est inférieur, égal ou supérieur.**
- Implémenter Comparable<T> donne à la classe un unique ordre « naturel ».
- Collections.sort(), Arrays.sort() et TreeSet/TreeMap (séance 4) l'utilisent automatiquement.
- Convention : si compareTo retourne 0, equals() devrait idéalement retourner true.

Q3.3

Comparator — Ordres personnalisés

```
public class ByNameComparator implements
Comparator<Employee> {
    @Override
    public int compare(Employee a, Employee b) {
        return a.getName().compareTo(b.getName());
    }
}
staff.sort(new ByNameComparator());

// style moderne avec lambda (Java 8+) : même idée, moins
de code
staff.sort(Comparator.comparing(Employee::getName));
staff.sort(Comparator.comparingDouble(Employee::getSalary).
reversed());
```

- **Comparator est externe à la classe : Employee n'a pas besoin d'être modifiée pour être triée autrement.**
- Vous pouvez définir autant de Comparators que nécessaire : par nom, salaire, date d'embauche…
- Comparator.comparing(...) est l'écriture moderne : un Comparator en une ligne construit à partir d'une référence de méthode.

STRATÉGIES DE COMPARAISON

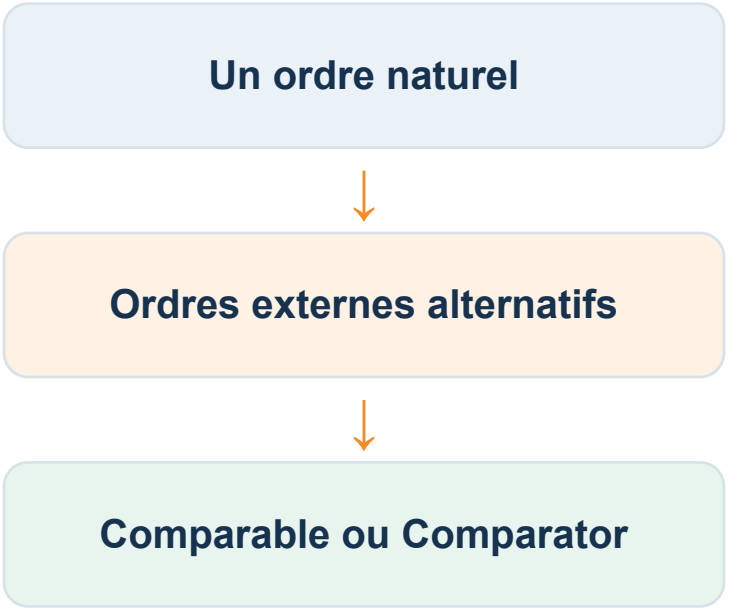
Comparable définit un ordre ; Comparator fournit des alternatives

Q3.2 / Q3.3

Comparable est implémenté par la classe elle-même.

Comparator est externe et peut être créé plusieurs fois pour différents besoins.

Aucune des deux stratégies ne doit soustraire des valeurs numériques pour les comparer.



GUIDE DE DÉCISION

Comparable ou Comparator — Quel choix ?

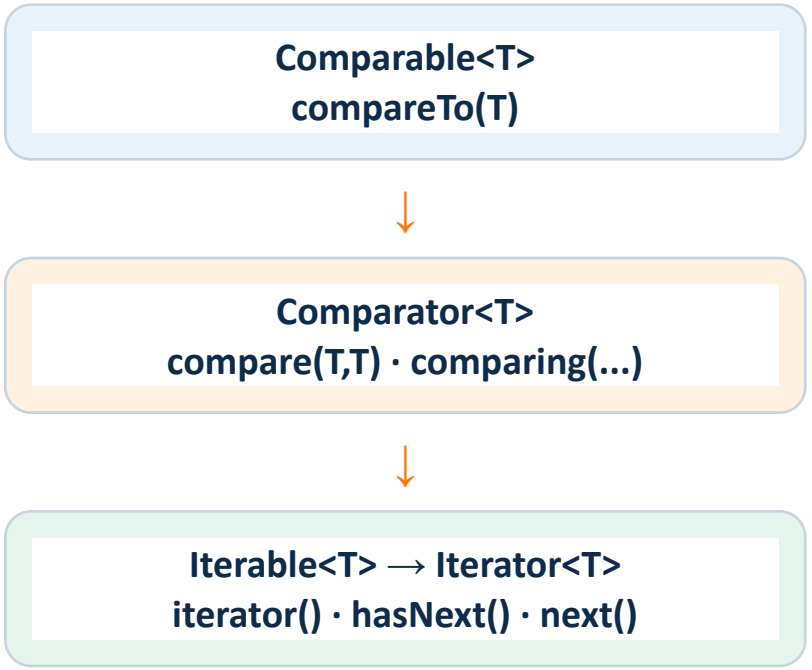
Situation	Choix
La classe possède un ordre naturel évident (ex. Fraction par valeur)	Comparable<T> — implémenté dans la classe
Plusieurs ordres sont nécessaires pour une même classe	Comparator<T> — un par ordre, externe à la classe
Le code source de la classe ne peut pas être modifié	Comparator<T> — toujours externe
La logique de comparaison doit rester hors de la classe métier	Comparator<T>

Interfaces (Java 8/9) : les méthodes default et private permettent de fournir une implémentation fonctionnelle ; les méthodes static jouent le rôle d'utilitaires, comme Comparator.comparing(...). En cas de conflit entre une méthode default et une méthode de classe héritée, la méthode de classe est prioritaire.

Une interface est d’abord un contrat officiel à lire

Dans la Javadoc, repérez :

la méthode abstraite principale ;
les paramètres de type ;
les méthodes default et static ;
les contraintes indiquées dans le contrat.



JDK 21 · Comparable / Comparator / Iterator

Utiliser explicitement un Iterator

```
List<String> names = new ArrayList<>(List.of("Ana", "Leo",
"Mia"));

Iterator<String> it = names.iterator();
while (it.hasNext()) {
    String name = it.next();
    if (name.startsWith("L")) {
        it.remove();          // seule méthode sûre pour
supprimer pendant l'itération
    }
}
// for (String n : names) { ... } correspond à cette
boucle, écrite automatiquement
```

- **Iterator<T> expose trois méthodes : hasNext(), next() et remove().**
- La boucle for-each est une simplification syntaxique de cette boucle while.
- Supprimer avec list.remove(...) pendant un for-each provoque ConcurrentModificationException ; it.remove() est l'alternative sûre.

PROTOCOLE D’ITÉRATION

Un itérateur sépare le parcours de la représentation

Le client pose seulement deux questions :

hasNext() : un autre élément est-il disponible ?
next() : retourner l’élément et avancer.

La collection conserve sa structure interne masquée.



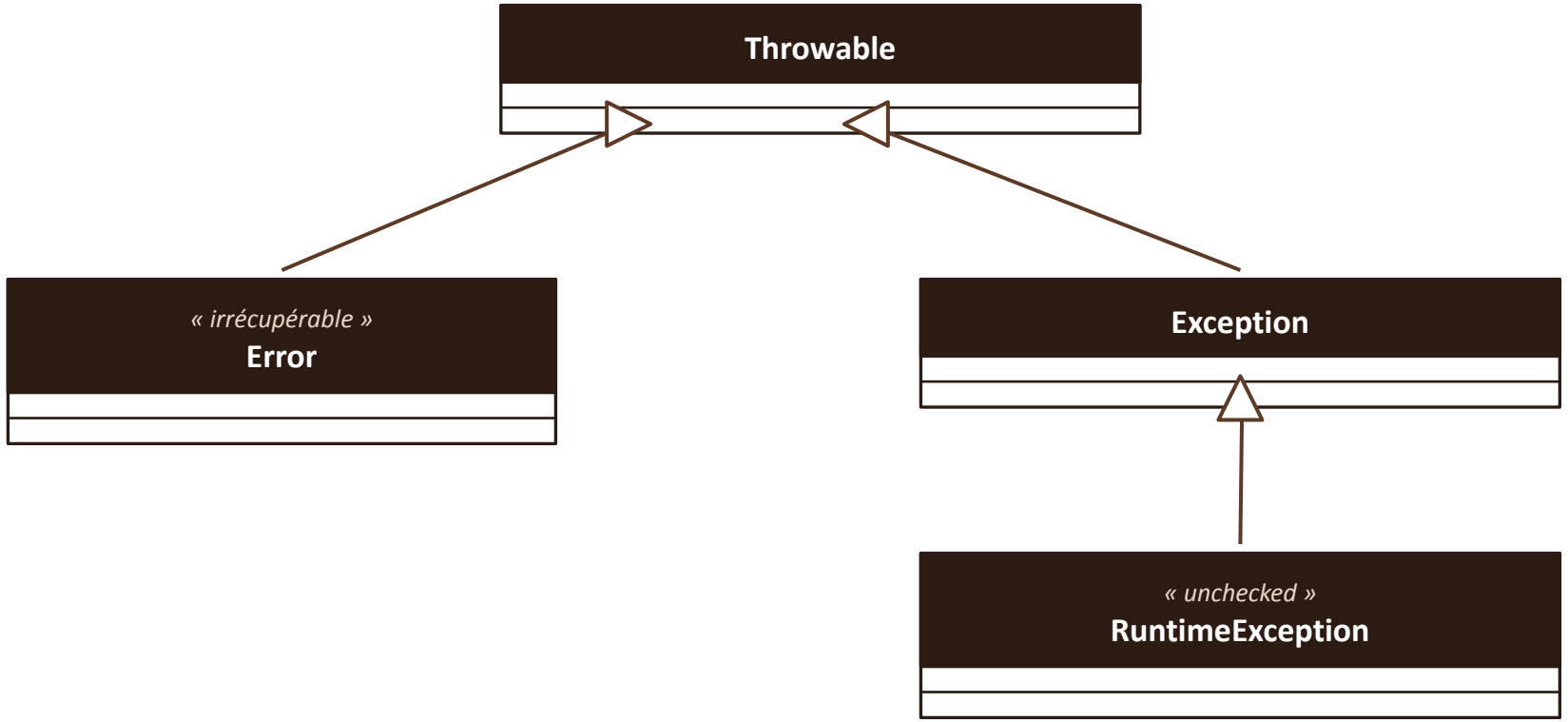
Écrire votre propre classe Iterable

```
public class Playlist implements Iterable<String> {
    private String[] tracks;
    private int size;
    // constructeur et add(...) omis

    @Override
    public Iterator<String> iterator() {
        return new Iterator<String>() {
            private int index = 0;
            @Override public boolean hasNext() { return
index < size; }
            @Override public String next() {
                if (!hasNext())
                    throw new NoSuchElementException();
                return tracks[index++];
            }
        };
    }
}
for (String track : new Playlist(...))
System.out.println(track);
```

- **implements Iterable<T> exige une seule méthode : iterator().**
- Ici, iterator() retourne une classe interne anonyme qui implémente Iterator<String> avec hasNext() et next(). remove() est facultative et l'implémentation par défaut lève UnsupportedOperationException.
- Dès qu'une classe est Iterable, elle peut être utilisée directement dans une boucle for-each, comme une ArrayList.

Exceptions checked et unchecked



Error — hors du contrôle de l'application, par exemple OutOfMemoryError : ne cherchez pas à les intercepter.

Exceptions checked (Exception hors RuntimeException), par exemple IOException : le compilateur impose de les traiter ou de les déclarer.

Exceptions unchecked (RuntimeException), par exemple NullPointerException : elles signalent souvent un bug ; leur traitement n'est pas imposé par le compilateur.

PROPAGATION D’UNE EXCEPTION

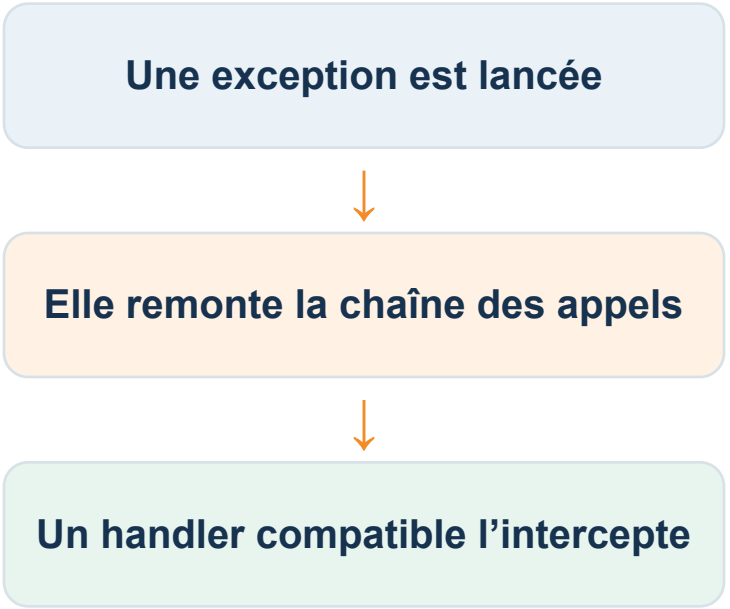
Une exception se propage jusqu’à un handler compatible

Q3.4

Lancer une exception interrompt le déroulement normal.

Chaque appelant peut traiter l’exception ou la laisser se propager.

Utilisez finally ou try-with-resources pour garantir le nettoyage sur tous les chemins.



Q3.4

Lancer et intercepter des exceptions

```
public void withdraw(int amount) {
    if (amount > balance) {
        throw new IllegalStateException("Solde
insuffisant");
    }
    balance -= amount;
}

try {
    account.withdraw(500);
} catch (IllegalStateException e) {
    System.out.println("Retrait impossible : " +
e.getMessage());
} finally {
    System.out.println("Tentative terminée."); // exécuté à
la sortie normale du try/catch
}
```

- **Pour lancer une exception : choisir ou écrire une classe d'exception → créer une instance → la lancer avec throw.**
- Les blocs catch sont testés de haut en bas : placez les types les plus spécifiques en premier.
- finally s'exécute avec ou sans exception : il convient aux opérations de nettoyage.

Q3.4

try-with-resources et exceptions personnalisées

```
try (Scanner sc = new Scanner(new File("data.txt"))) {
    while (sc.hasNextLine()) process(sc.nextLine());
}   // sc.close() est appelé automatiquement, même en cas
d'exception

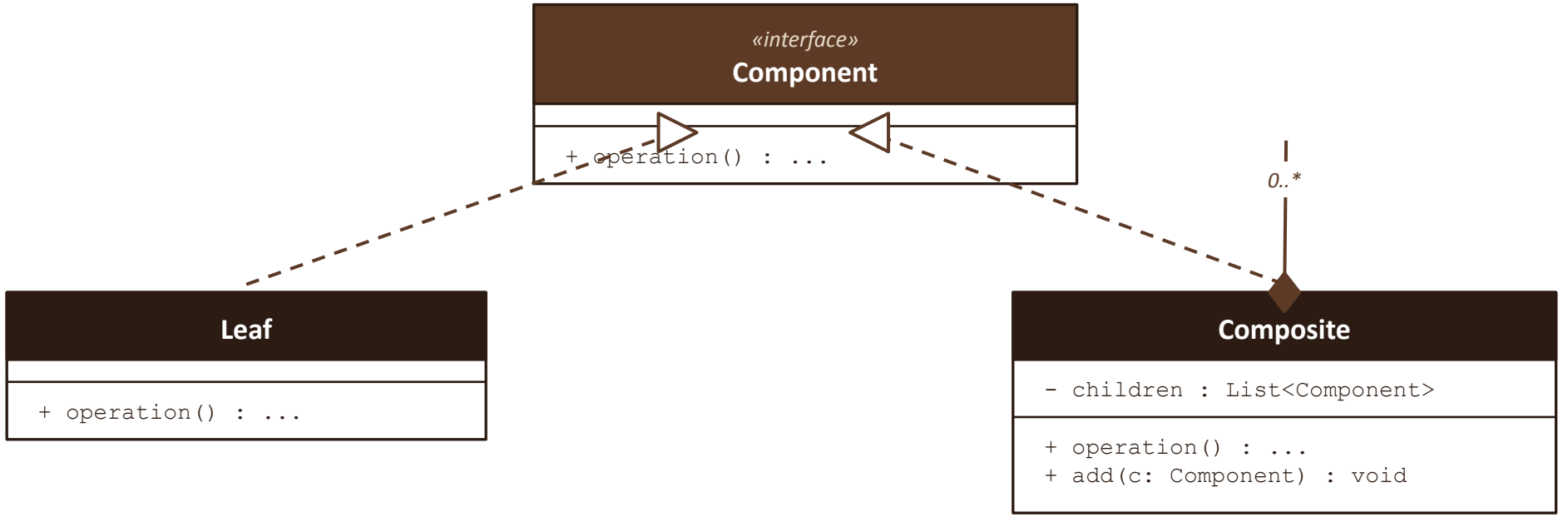
public class InvalidAmountException extends Exception {
    public InvalidAmountException(String message) {
        super(message);
    }
}
public void deposit(int amount) throws
InvalidAmountException {
    if (amount < 0)
        throw new InvalidAmountException("Le montant ne
peut pas être négatif");
    // poursuivre le dépôt
}
```

- **try-with-resources ferme automatiquement toute ressource AutoCloseable : plus d'oubli de close().**
- Étendez Exception pour une exception personnalisée checked que l'appelant doit traiter, ou RuntimeException pour une exception unchecked.
- Bonne pratique : échouer rapidement, utiliser des messages précis et ne jamais ignorer silencieusement une exception avec un catch vide.

Qu’est-ce qu’un design pattern ?

- Un design pattern est une solution nommée et réutilisable à un problème de conception récurrent. Ce n’est pas une bibliothèque à importer, mais une structure à reconnaître et à implémenter.
- Sa valeur tient au vocabulaire : dire « utilisons Composite ici » communique immédiatement une structure entière à un autre développeur.
- Les patterns sont traditionnellement regroupés en trois familles :
  - Creational — création des objets (ex. Factory, Builder)
  - Structural — composition des classes et objets (ex. Composite, Decorator)
  - Behavioral — communication et partage des responsabilités entre objets (ex. Iterator et Strategy)
- Nous étudions aujourd’hui un pattern structural : Composite, car vous en aurez rapidement besoin.

Composite — Intention et structure



Intention : traiter un objet individuel (Leaf) et un groupe d'objets (Composite) via la même interface, sans que le code client ait à les distinguer.

À utiliser lorsque les données forment naturellement un arbre : un conteneur pouvant contenir d'autres conteneurs, comme des dossiers, un organigramme ou des menus imbriqués.

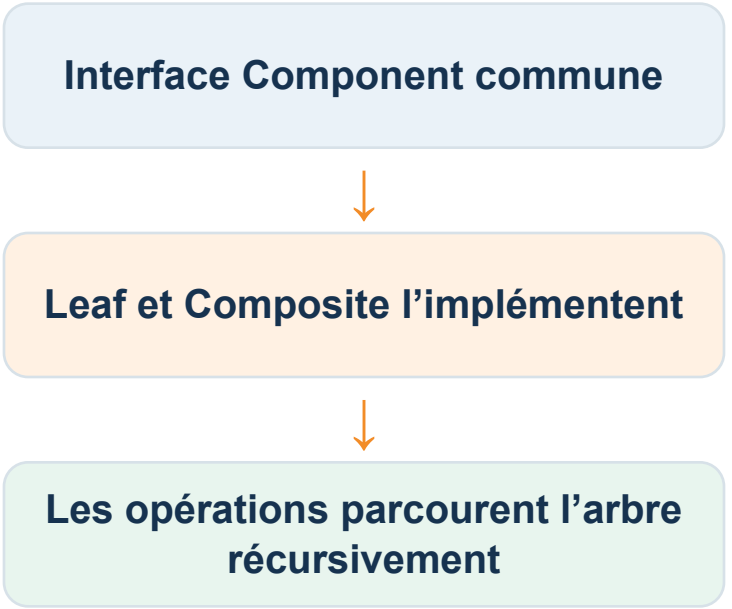
PATTERN COMPOSITE

Composite traite uniformément les objets individuels et les groupes

Les objets Leaf et Composite implémentent la même interface.

Le Composite délègue récursivement l'opération à ses enfants.

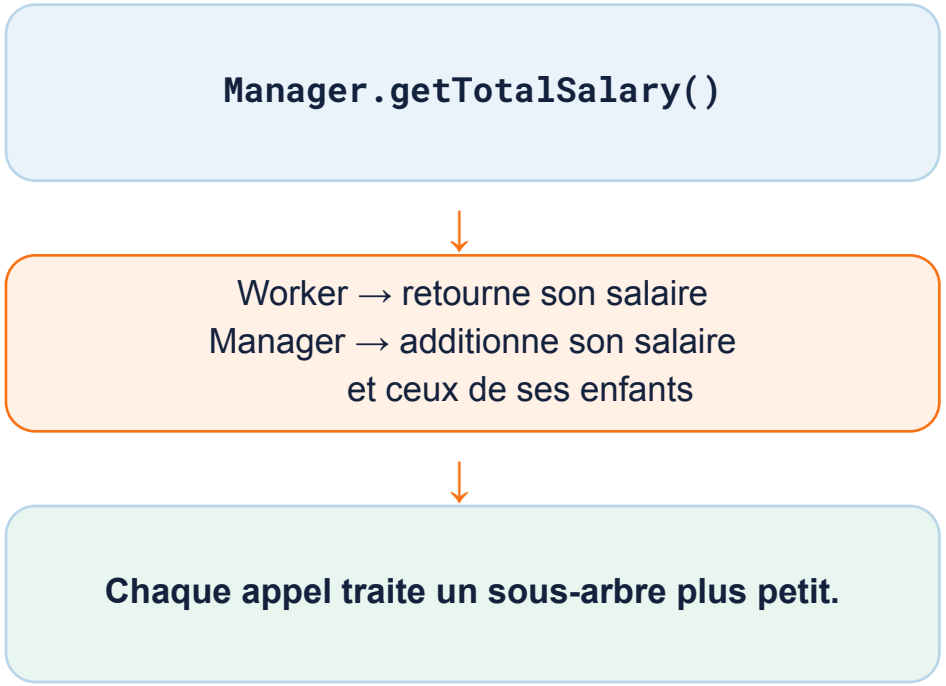
Le client n'a pas besoin de tester les types.



Un même calcul descend dans l’arbre

La récursion est un appel d’une méthode à elle-même sur un sous-problème plus petit.

Dans Composite, chaque groupe demande le résultat à ses enfants. Le parcours s’arrête naturellement sur les feuilles, qui ne contiennent aucun enfant.



Composite — Exemple guidé

```
public abstract class Employee {
    protected String name;
    public abstract double getTotalSalary();
}

public class Worker extends Employee {      // Leaf
    private double salary;
    public double getTotalSalary() { return salary; }
}

public class Manager extends Employee {     // Composite
    private double salary;
    private List<Employee> reports = new ArrayList<>();

    public void addReport(Employee e) { reports.add(e); }

    public double getTotalSalary() {
        double total = salary;
        for (Employee e : reports)
            total += e.getTotalSalary();    // récursion
        return total;
    }
}
```

- **Manager contient une liste d'Employee qui peut elle-même contenir d'autres Managers.**
- getTotalSalary() s'appelle récursivement dans l'arbre, quelle que soit sa profondeur.
- Le File / Folder du TD3 possède la même structure : Folder contient des Files, qui peuvent eux-mêmes être des Folders.
- Iterator est lui aussi un design pattern : vous en avez déjà implémenté un.

SYNTHÈSE

À retenir

- ✓ Une interface est un contrat CAN-DO ; une classe peut en implémenter plusieurs, contrairement à l'héritage de classe unique.
- ✓ Comparable donne à une classe un ordre naturel ; Comparator permet de définir autant d'ordres externes que nécessaire.
- ✓ Iterator/Iterable permettent de parcourir une collection, y compris personnalisée, sans exposer sa structure interne.
- ✓ Les exceptions checked doivent être déclarées ou traitées. Les exceptions unchecked signalent souvent une erreur de programmation ou de contrat, mais peuvent aussi représenter une entrée ou un état irrécupérable. finally et try-with-resources garantissent le nettoyage.
- ✓ Un design pattern est une structure de solution nommée et réutilisable. Composite permet de traiter uniformément un élément et un groupe dans un arbre.

PASSAGE À LA PRATIQUE

Pour la suite

TD3

Dans le TD3, vous trierez des objets avec Comparator, parcourrez une liste avec un Iterator explicite, gérerez des exceptions et construirez une hiérarchie File/Folder qui applique directement le pattern Composite vu avec l'organigramme. Après cette séance, seules les classes concrètes du TD3 seront nouvelles.